Open in app ↗

Medium Search Play Games Write 3

Welcome Offer Access to everything. Now 30% off. Upgrade now

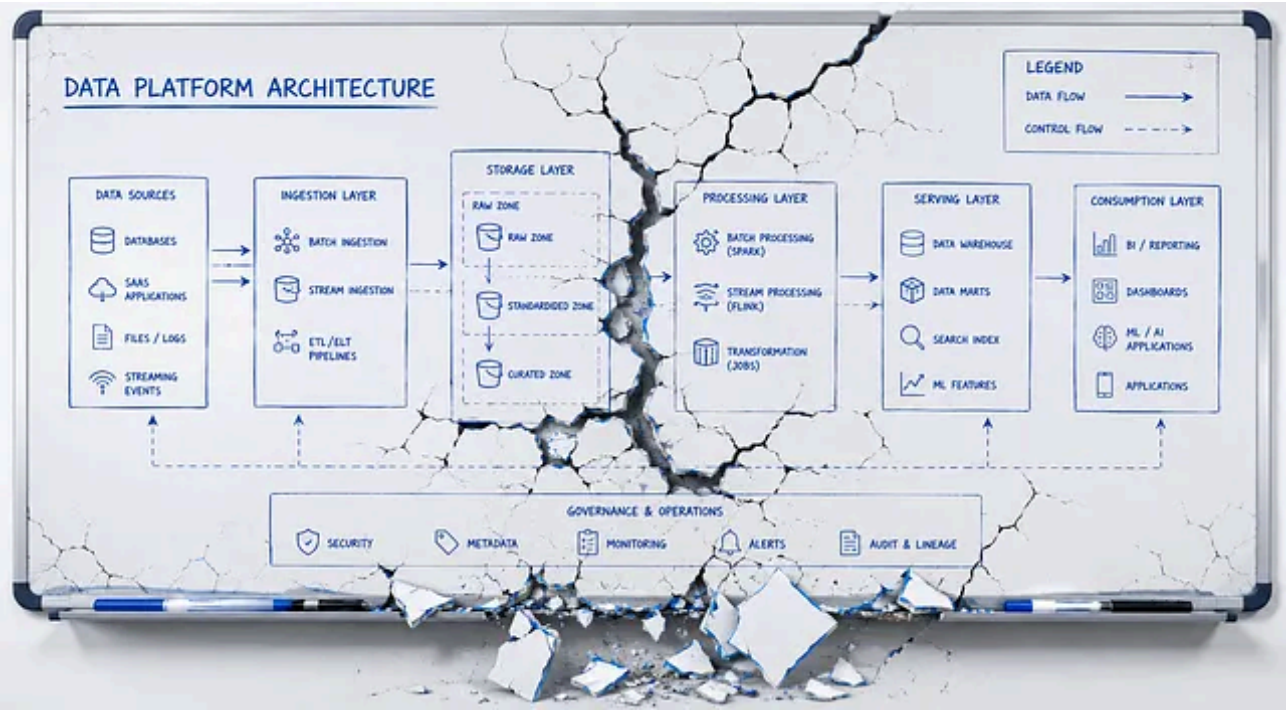
Data Engineering + Big Data + Data Engineer + Design Systems +

System Design Interview +

In an Interview I Was Asked: “Can You Create a Project Architecture Which Will Fail 100% For Sure?”



Anurag Sharma 17 min read · Sep 14, 2026



An interviewer once asked me to design a big data architecture guaranteed to fail — 100%, no exceptions. That question turned out to be one of the smartest ways to teach architecture I have ever encountered. This blog walks through every layer of a big data system — ingestion, storage, processing, orchestration, data quality, governance, cost, and team ownership — and names the exact decision at each layer that guarantees collapse. Then it flips every single one into the fix. You will leave with a working mental checklist of anti-patterns that’s more useful than any “best practices” list, because you’ll understand *why* each one kills projects.

Who should read this?

- Are you designing or reviewing a big data platform right now, and want to sanity-check it against real failure modes?
- Are you preparing for a data engineering or architecture interview and want an answer better than reciting best practices?
- Have you inherited a data platform where “nobody quite trusts the dashboards anymore,” and you’re trying to figure out why?
- Are you a data leader trying to explain to stakeholders why “just add more engineers” won’t fix a broken platform?
- Are you simply curious how small, individually reasonable shortcuts compound into full-blown system failure?

If any of the questions are YES, then this blog is for you.

The Question That Stopped Me Cold

I have sat through dozens of data engineering interviews. I have been asked to design real-time pipelines, defend my choice of columnar vs row-based

storage, and whiteboard a lambda architecture from memory. But I had never been asked this:

> “Design me a big data architecture that is guaranteed to fail. Not likely to fail — guaranteed. 100%.”

My first instinct was to laugh it off as a trick question. My second instinct — the correct one — was to recognise it as one of the smartest interview questions I had ever heard. Because if you can articulate, with precision, exactly how to destroy a system, you understand that system far better than someone who can only recite best practices from a blog post.

This is the concept of **inversion**, borrowed from Charlie Munger’s “invert, always invert.” Instead of asking “how do I succeed?” you ask “how do I guarantee failure?” — and then you do the opposite. It’s a powerful mental model because failure modes are often more concrete, more memorable, and more instructive than success stories. Success in big data architecture is quiet and unremarkable. Failure is loud, expensive, and educational.

So that is what this blog is. I am going to walk you through the complete blueprint for an architecture that fails 100% of the time — layer by layer, decision by decision — and then flip each one to show you what a resilient, production-grade big data architecture actually looks like. By the end, you will have a mental checklist of “100% don’ts” that is more useful than any “10 best practices” listicle, because you will understand *why* each one is fatal.

Part 1: The Setup — Why Inversion Works

Before we build our failure machine, it is worth understanding why this framing is so effective, especially in an interview context.

When you ask someone to design a *successful* system, they default to pattern-matching against architectures they have seen work — Kappa architecture, medallion architecture, data mesh, whatever’s trendy. They recite. They do not necessarily reveal judgment.

When you ask someone to design a system that **fails**, they have to reason about causality. They have to know which decisions compound into disaster, which failures are silent versus loud, and which mistakes are recoverable versus catastrophic. It exposes whether someone has actually operated a system at 3 am during an incident, or whether they have only ever built demos.

There is also a deeper truth here specific to big data: most big data projects do not fail because of a single catastrophic bug. They fail through **slow, compounding neglect** — a schema that drifts, a pipeline that silently drops 2% of records, a cost curve that grows unnoticed until finance intervenes, a data team that stops trusting its own dashboards. Guaranteed failure is not about writing bad code. It is about making a series of individually defensible decisions that interact catastrophically.

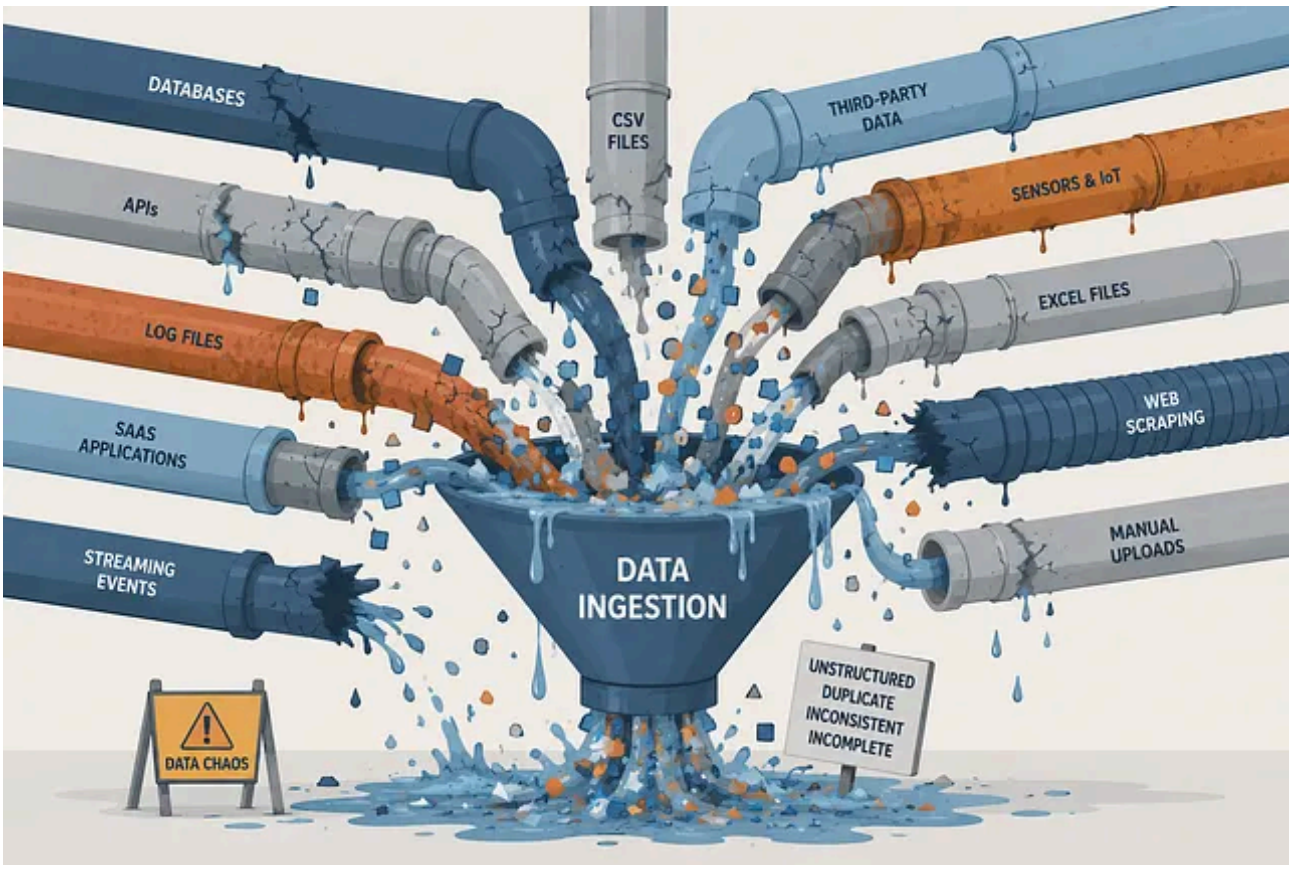
Part 2: The Guaranteed-Failure Architecture, Layer by Layer

2.1 Requirements & Scoping — Don’t Do This

The 100% failure move: Skip requirements gathering entirely. Let the loudest stakeholder define “big data” as “all of our data, forever, just in case.” Never define what questions the system needs to answer. Never define latency requirements, freshness SLAs, or who the consumers are. Start building infrastructure before a single business question is written down.

Why it guarantees failure: Big data architecture is not a technology decision; it is an information-flow decision. Without a defined consumer and a defined question, you cannot make a single correct decision about storage format, partitioning, or processing paradigm. Teams that skip this step build “data lakes” that are actually data swamps — enormous, expensive, and unable to answer a single business question quickly because nothing was ever organised around an answer.

What to do instead: Start with 3–5 concrete analytical or operational questions the system must answer, along with explicit freshness and latency requirements (batch daily? streaming sub-second? near-real-time in minutes?). Architecture should be a derived consequence of these requirements, not a starting point.



2.2 Data Ingestion — Don’t Do This

The 100% failure move:

- Ingest everything from every source with no schema validation at the point of entry.
- Treat schema-on-read as a philosophy rather than a fallback — assume you will “figure out the structure later.”
- Never version your ingestion contracts. Let upstream teams change field names, types, and nesting whenever they want, with zero notification.
- Build point-to-point custom ingestion scripts for every single source instead of a standardised ingestion framework. 200 sources, 200 bespoke scripts, no shared error handling.
- Ignore idempotency. If a job fails halfway through, just rerun it and let duplicates pile into downstream tables.

Why it guarantees failure: Ingestion is the foundation. Every downstream layer inherits its sins. Schema drift without contracts causes silent breakage — a rename from `user_id` to `userId` upstream won't throw an error; it'll just quietly produce a new column full of nulls in your fact table while the old column keeps existing with stale values, and nobody notices until a quarterly report is wrong. Non-idempotent ingestion means every failure recovery introduces duplicate or missing data, and duplicates in a 50-billion-row fact table are a nightmare to detect and near-impossible to cleanly reverse.

What to do instead:

- Enforce schema contracts (e.g., via a schema registry with Avro/Protobuf, or contract testing between producer and consumer teams).

- Build a standardised, config-driven ingestion framework rather than one-off scripts per source.
- Make every ingestion job idempotent — use deterministic keys, upserts, or write-once partitioning by ingestion timestamp so reruns are safe.
- Version schema changes and require backwards-compatible evolution (additive fields, not silent renames/type changes).

2.3 Storage & Modelling — Don’t Do This

The 100% failure move:

- Store everything as one giant JSON blob “for flexibility.”
- Pick a storage format (say, row-oriented CSV) without any relationship to your query patterns, and never revisit it.
- Never partition anything. One giant table, unpartitioned, scanned in full on every query.
- Mix raw, cleaned, and business-logic-transformed data in the same tables with no separation of layers.
- Have no data modelling standard — every team invents its own naming conventions, grain, and slowly-changing-dimension strategy (or none at all).

Why it guarantees failure: Storage decisions are the hardest to reverse in a data system — migrating petabytes of misformatted data later is enormously expensive. Unpartitioned tables mean every query becomes a full scan, so costs and latency grow linearly (or worse) with data volume, and what was fast at 10GB becomes unusable at 10TB. Mixing raw and transformed data destroys your ability to reprocess history when business logic changes, and

destroys your ability to trust which “version of the truth” a downstream consumer is reading.

What to do instead:

- Adopt a layered storage strategy — raw/bronze (immutable, as-ingested), cleaned/silver (validated, deduplicated, conformed), and business/gold (aggregated, modelled for consumption). This is the medallion pattern for a reason: it isolates blast radius and enables reprocessing.
- Choose columnar formats (Parquet, ORC) for analytical workloads, and partition based on actual query predicates (commonly by date, sometimes additionally by a high-cardinality-but-frequently-filtered key).
- Establish a shared data modelling standard (dimensional modelling, Data Vault, or whatever fits) and enforce it through review, not just documentation nobody reads.

2.4 Processing & Compute — Don’t Do This

The 100% failure move:

- Choose your processing engine based on what’s trendy on Linkedin, Twitter/X this month, not on your actual workload shape.
- Write business logic directly inside orchestration DAG definitions, so there’s no way to unit test transformation logic outside of a full pipeline run.
- Never separate compute from storage — tightly couple your processing cluster’s lifecycle to your data’s lifecycle so you can’t scale one without the other.

- Hard-code cluster sizes for “peak load” and run them 24/7, whether or not there’s actually data to process.
- Allow the same transformation logic to be duplicated (and subtly diverge) across three different pipelines because nobody built shared, tested transformation libraries.

Why it guarantees failure: Logic embedded inside orchestration code is untestable, and untested transformation logic is where silent correctness bugs live for months. Duplicated logic across pipelines guarantees that “revenue” computed in the marketing pipeline eventually stops matching “revenue” computed in the finance pipeline — and now you have a credibility crisis, not just a bug. Coupling compute and storage lifecycles, plus static “peak load” provisioning, is how a company burns 3x its cloud budget running idle clusters against a batch job that only needs to run for 40 minutes a day.

What to do instead:

- Choose your processing engine (Spark, Flink, dbt, warehouse-native SQL, whatever) based on latency requirements and team skill, not hype.
- Keep transformation logic as testable, versioned code (with unit tests) — orchestration should call logic, not contain it.
- Decouple compute from storage; use elastic/serverless compute where feasible so cost scales with actual usage.
- Build a shared library of core business metric definitions used by every downstream pipeline — a single source of truth for “how do we calculate revenue.”

2.5 Orchestration & Pipeline Design — Don’t Do This

The 100% failure move:

- Chain jobs together via cron and hope, with no dependency graph, no retries, and no idea what happens if job 3 of 12 fails.
- Make every pipeline a monolith — one giant DAG that does ingestion, transformation, and serving in a single unbroken chain with no checkpoints.
- Never implement backfill capability. If historical data needs reprocessing (and it always eventually does), there is literally no mechanism to do it without manual, error-prone one-off scripts.
- Have pipelines silently swallow errors and continue — log a warning nobody reads, then proceed as if everything is fine.

Why it guarantees failure: Big data pipelines fail constantly — that’s not a hypothetical; it’s an operating assumption. A source will time out. A schema will drift. A cluster will get preempted. An architecture that has no retry logic, no checkpointing, and no backfill capability treats every one of these routine events as an emergency requiring manual intervention, which does not scale past a handful of pipelines. Silent error swallowing is the most insidious of all: the pipeline “succeeds” every day while quietly corrupting downstream data, and by the time someone notices, months of reports have been wrong.

What to do instead:

- Use a proper orchestrator (Airflow, Dagster, Prefect, or your cloud-native equivalent) with explicit dependency graphs, retry policies, and alerting on failure.

- Break monolithic DAGs into modular, independently-testable stages with checkpoints so a mid-pipeline failure doesn’t force a full restart from scratch.
- Design idempotent, parameterized pipelines from day one so backfills are a routine operation, not a fire drill.
- Fail loudly. A pipeline that errors visibly is infinitely cheaper than one that succeeds silently while producing garbage.



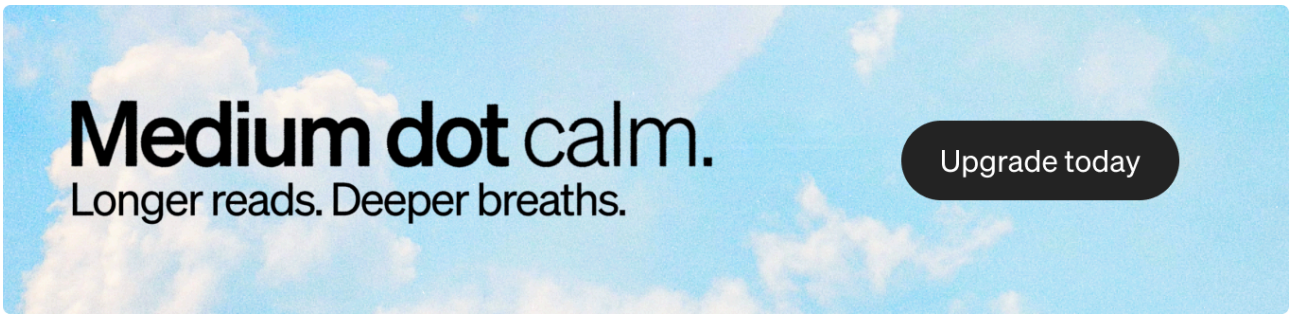
2.6 Data Quality & Observability — Don’t Do This

The 100% failure move:

- Have zero automated data quality checks. Trust that if the pipeline “ran successfully,” the data must be correct.

- Never monitor data freshness, volume anomalies, or distribution drift — only monitor infrastructure metrics (CPU, memory) and assume that’s equivalent to monitoring data health.
- Let dashboards be the first line of quality detection — i.e., let business users discover data problems before your team does.
- Have no data lineage. When a number looks wrong, there’s no way to trace which upstream source or transformation produced it.

Why it guarantees failure: This is, in my experience, the single most common real-world cause of big data project failure — not architecture complexity, but the absence of a nervous system. A pipeline can run green every day, on schedule, with no errors thrown, and still be feeding a dashboard 40% too-low numbers because of a silent join fan-out. Without lineage, root-causing that takes days instead of minutes, and every incident erodes stakeholder trust in the platform a little more, until they start keeping their own spreadsheets “just to double-check” — which is the moment your data platform has functionally failed regardless of what the uptime metrics say.



What to do instead:

- Implement automated data quality checks at every layer (row counts, null rates, schema conformance, referential integrity, statistical distribution checks) using something like Great Expectations, dbt tests, or Deequ.

- Monitor data-specific signals, not just infra signals: freshness (is data as recent as expected?), volume (did row counts change unexpectedly?), and distribution (did a metric jump 300% overnight?).
- Invest in lineage tooling (OpenLineage, dbt’s built-in lineage, or platform-native equivalents) so root cause analysis is measured in minutes.
- Treat “your team finds the bug before the business does” as a core SLA, not a nice-to-have.

2.7 Governance, Security & Access Control — Don’t Do This

The 100% failure move:

- Give every engineer admin access to every dataset, “to move fast.”
- Never classify data sensitivity — treat PII, financial data, and public marketing metrics identically.
- Skip encryption at rest and in transit because “it’s an internal network.”
- Have no audit trail of who accessed or modified what data.
- Bolt on compliance (GDPR, HIPAA, SOC 2, whatever applies) at the very end, right before an audit, instead of designing for it from the start.

Why it guarantees failure: This category of failure is different from the others — it doesn’t just degrade the system; it can end the project (or the company) outright via a regulatory fine, a breach, or a failed audit that blocks a critical customer contract. Retrofitting governance onto a system that was never designed with data classification or access boundaries is enormously expensive and often incomplete, because sensitive fields have already propagated into dozens of downstream tables, caches, and exports that nobody tracked.

What to do instead:

- Classify data sensitivity at ingestion and propagate that classification through lineage metadata automatically.
- Implement role-based or attribute-based access control from day one, scoped to the principle of least privilege.
- Encrypt data at rest and in transit as a default, not an exception.
- Maintain audit logs of access and modification.
- Design for your compliance regime from the start — it’s a requirement, not a checkbox added later.

2.8 Scalability & Cost Management — Don’t Do This

The 100% failure move:

- Provision for today’s data volume with no plan for 10x growth, and also over-provision “just in case” so cost balloons before volume ever justifies it — do both, inconsistently, depending on who asked.
- Never tag resources by team, project, or purpose, so cost attribution is impossible, and nobody feels ownership over their spend.
- Store everything in the highest-performance (and most expensive) storage tier forever, including data nobody has queried in two years.
- Let query patterns go unreviewed — allow analysts to run `SELECT *` full-table scans on multi-terabyte tables as a matter of routine.

Why it guarantees failure: Big data projects are frequently killed not by a technical failure but by a financial one — the CFO sees a cloud bill that’s grown 400% in a year with no corresponding business value delivered, and pulls funding. Unmanaged cost is a silent, compounding failure mode exactly like unmanaged data quality: nothing looks broken from an engineering dashboard right up until the budget conversation.

What to do instead:

- Build in autoscaling and usage-based compute so cost tracks actual demand, plus capacity planning tied to realistic growth projections.
- Tag every resource for cost attribution, and put spend dashboards in front of the teams generating it.
- Implement storage tiering — hot, warm, cold/archive — with lifecycle policies that automatically age out infrequently-accessed data.
- Review and optimize expensive query patterns; put guardrails (like cost limits or required partitioning filters) in front of ad hoc query tools.

2.9 Team, Process & Ownership — Don’t Do This

The 100% failure move:

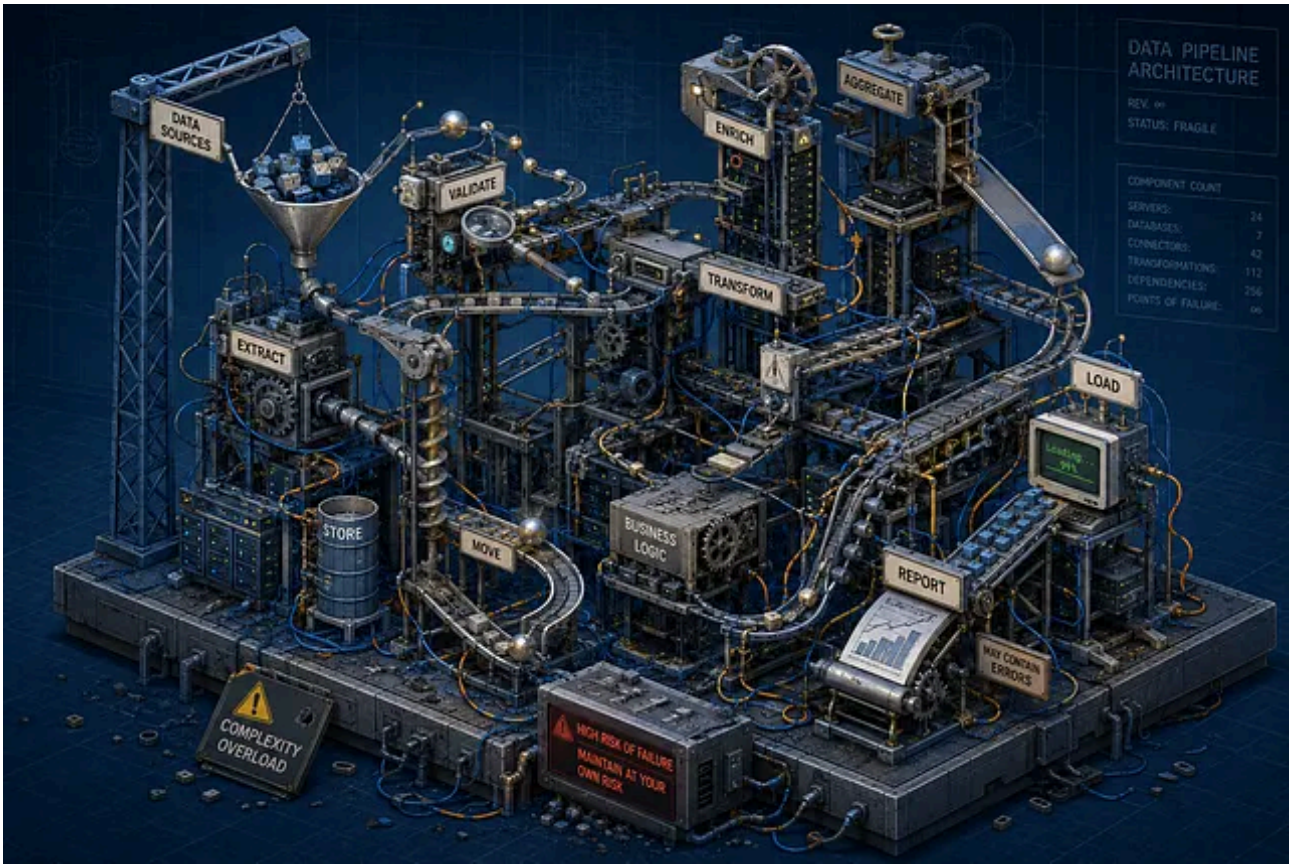
- Have no clear ownership of pipelines — when something breaks, it’s “everyone’s” and therefore no one’s problem.
- Let tribal knowledge live only in one senior engineer’s head, with zero documentation, and then let that engineer leave.
- Never do post-incident reviews. When a pipeline fails catastrophically, patch it silently and move on without writing anything down.

- Treat the data platform as a purely engineering initiative — never involve the business stakeholders who actually consume the data in architecture decisions.

Why it guarantees failure: Technology doesn’t fail data projects nearly as often as organizations do. A perfectly designed architecture with no clear ownership model degrades within a year, because ambiguous ownership means nobody prioritizes maintenance, nobody notices slow-burning issues, and every incident becomes an archaeology exercise. Bus-factor-of-one systems are a ticking clock, not a hypothetical risk. And a platform designed without its actual consumers in the room reliably produces technically elegant systems that answer nobody’s real question — technically successful, functionally dead.

What to do instead:

- Assign clear, named ownership for every pipeline and dataset, with an on-call or support rotation.
- Document architecture decisions (ADRs), runbooks, and tribal knowledge as a continuous practice, not a one-time effort.
- Run blameless post-incident reviews and actually act on the findings.
- Involve business stakeholders in requirements and review cycles throughout the project, not just at kickoff and final demo.



Part 3: Assembling the Complete Failure Machine

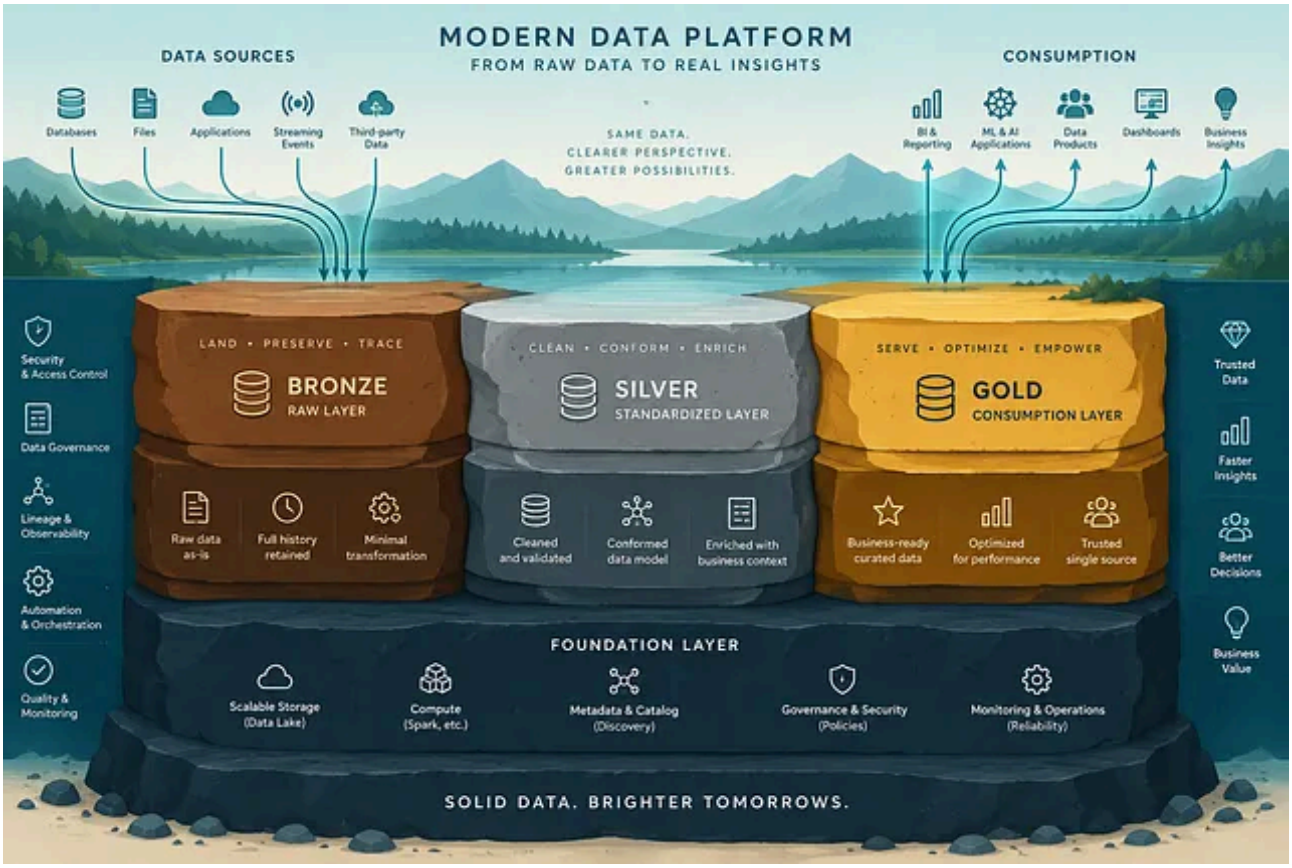
If you take every single “100% failure move” above and stack them together, here’s what the resulting architecture actually looks like, described end to end:

Data arrives from two hundred unversioned sources into one enormous unpartitioned table with no schema validation. A single senior engineer’s cron jobs pull, transform, and load everything in one giant nightly script with no retries, no checkpoints, and no idempotency — so failures either silently skip data or silently duplicate it. Business logic is written directly inside the scheduler config, duplicated slightly differently in four other places. There are no data quality checks, no lineage, no freshness monitoring, so nobody knows anything is wrong until an executive dashboard shows revenue is down 60% quarter over quarter, and someone

finally investigates. Every engineer has admin access to every table, including tables full of unmasked customer PII, with no audit log. The whole thing runs on statically-provisioned, always-on compute sized for a “just in case” peak load that never materializes, quietly costing 5x what it should. And when it all breaks — which it does, repeatedly — there’s no runbook, no documented owner, and the one person who understood the system left the company eight months ago.

That system doesn’t fail because of one catastrophic event. It fails the way most real big data projects actually fail: gradually, quietly, and then all at once, when trust collapses, and the business stops believing the numbers.

That’s the real insight the interview question was testing. **Big data failure is rarely a single bad decision. It’s a systemic property that emerges from many individually plausible shortcuts compounding across layers that don’t talk to each other.**



Part 4: What a Resilient Architecture Actually Looks Like

Flip every single decision above, and you get something close to a modern, boring, resilient big data architecture:

1. **Requirements-first.** Concrete questions, defined freshness/latency SLAs, identified consumers before infrastructure decisions.
2. **Contract-governed ingestion.** Schema registry, versioned contracts, standardized ingestion framework, idempotent writes.
3. **Layered storage.** Bronze/silver/gold (or equivalent) separation, columnar formats, partitioning aligned to real query patterns.
4. **Testable, decoupled processing.** Business logic as versioned, unit-tested code; compute decoupled from storage; elastic scaling.

5. **Robust orchestration.** Modular DAGs, retries, checkpoints, idempotent and backfill-friendly design, loud failure over silent failure.
6. **Observability as a first-class citizen.** Automated data quality checks, freshness/volume/distribution monitoring, end-to-end lineage.
7. **Governance by design.** Data classification, least-privilege access, encryption, audit trails, compliance built in from day one.
8. **Cost-aware scaling.** Autoscaling compute, storage tiering, resource tagging, query guardrails.
9. **Clear ownership and process.** Named owners, documentation, blameless postmortems, business stakeholders embedded throughout.

None of this is exotic. None of it requires the newest tool on Hacker News. It requires discipline applied consistently across every layer, because the layers interact — a brilliant storage design doesn’t save you from an ungoverned ingestion layer, and perfect observability doesn’t save you from an architecture nobody owns.

Part 5: What the Interviewer Was Really Testing

Looking back, I don’t think that question was about big data at all — not really. It was about whether I understood that **architecture is a system of tradeoffs, and every tradeoff has a failure mode attached to it.** Anyone can list best practices. Fewer people can explain, precisely, what happens when you violate each one, and in what order the dominoes fall.

If you’re ever asked this question in an interview, I’d suggest answering it exactly the way this post is structured: pick the layers (ingestion, storage, processing, orchestration, quality, governance, cost, team), name the failure move at each layer, and explain the causal mechanism by which it kills the

project — not just “it would be bad,” but *how* it compounds. Then, unprompted, flip it to the fix. That shows you’re not reciting a list of anti-patterns you memorized — you understand the system well enough to break it on purpose and repair it on demand.

And if you’re building a real architecture right now, not just answering an interview question: take twenty minutes, go layer by layer through this list, and honestly ask which of these “100% failure moves” your current system is quietly making. In my experience, most production big data platforms have at least three or four of these lurking somewhere — not because the team is incompetent, but because these shortcuts are individually reasonable under deadline pressure. The discipline isn’t avoiding all of them once. It’s building the review habits and ownership structures that catch them before they compound.

That’s the architecture that fails 100% of the time — and now, hopefully, the one that doesn’t.

Conclusion

The interview question was never really about big data — it was about whether I understood architecture as a system of tradeoffs, where every shortcut has a failure mode attached to it. Big data projects rarely die from one catastrophic bug; they die from several individually defensible decisions compounding quietly across layers that don’t talk to each other, until trust in the data collapses. The fix isn’t exotic tooling — it’s discipline applied consistently: contracts at ingestion, layered storage, testable processing, observable pipelines, governance by design, cost awareness, and clear ownership. Master the failure modes, and the resilient architecture reveals itself as the natural opposite.

Where to go next

- **Audit your own stack:** pick one layer from this post (start with ingestion or observability — they’re usually the leakiest) and honestly check it against the failure modes listed.
- **Write your own “inversion” document:** For your current project, list the five ways it could guarantee failure, then assign an owner to address each one.
- **Read next:** dig deeper into data contracts, the medallion architecture pattern, and data observability tooling (Great Expectations, OpenLineage) — each gets a full post of its own soon.
- **Bring it to your team:** run this as a workshop exercise — have your team design a “guaranteed to fail” architecture together. It surfaces real, present-day risks faster than any retrospective.

Data Engineering + Big Data + Data Engineer + Design Systems +

System Design Interview +



Written by Anurag Sharma

149 followers · 8 following

Data Engineering Specialist with 11+ Exp. Passionate about optimizing pipelines, data lineage, and Spark performance and sharing insights to empower data pros!

Follow